

URL-Based Phishing Detection Using Machine Learning: Reproduction and Critical Evaluation

Executive Summary

This project reproduced and critically evaluated a machine learning-based phishing website detection system from the GitHub repository *URL-Based Phishing Detection Using Machine Learning*. The objective of the original project was to classify websites as either legitimate or phishing using URL and website-related features. Phishing detection is an important cybersecurity problem because phishing websites are frequently used to steal sensitive information such as usernames, passwords, and financial data.

The dataset contained 11,054 observations and 32 columns. After duplicate removal, the working dataset consisted of 5,849 unique observations. Multiple machine learning models were evaluated, including Logistic Regression, K-Nearest Neighbors, Naive Bayes, Decision Trees, Random Forests, Support Vector Machines, Multi-Layer Perceptrons, and Gradient Boosting.

The reproduction process confirmed that the dataset contains highly informative features for phishing detection. Correlation, crosstab, and feature importance analyses identified HTTPS, AnchorURL, and WebsiteTraffic among the most influential predictors. The reproduced experiments also confirmed the author's primary conclusion that Gradient Boosting achieved the best overall performance.

The reproduced Gradient Boosting model achieved an accuracy of 94.62%, Precision of 93.75%, Recall of 94.62%, F1-score of 94.18%, MCC of 0.892, and ROC-AUC of 0.992. These results indicate strong discrimination between phishing and legitimate websites and demonstrate the effectiveness of the selected feature set.

Although the primary findings were successfully reproduced, several methodological concerns were identified, including hardcoded file paths, documentation inconsistencies, missing random seeds, data leakage caused by scaling before the train-test split, and limited justification for certain preprocessing decisions. An additional experiment showed that SMOTE provided only modest performance improvements, suggesting that the strong results were primarily driven by the predictive quality of the features rather than oversampling.

Overall, the project demonstrates an effective application of machine learning to phishing website detection. The author's main conclusions were supported by the reproduced results; however, improvements in reproducibility practices, documentation quality, and methodological transparency would strengthen the reliability and scientific rigor of the work.

Topic

Phishing Website Detection

Problem Statement

Determine whether a URL belongs to a phishing website or a legitimate website using machine learning techniques.

Author's Main Claim

The repository claims that machine learning models can successfully detect phishing websites using URL-based features. Among the evaluated algorithms, the author concludes that the Gradient Boosting Classifier achieves the best performance, with an accuracy of approximately 94.5%.

Initial Questions to Investigate

1. Is the dataset clean and suitable for machine learning?
2. Are the selected features meaningful for phishing detection?
3. Is an accuracy of approximately 94% sufficient for this problem?
4. Are additional metrics such as Precision, Recall, and F1-score needed?
5. How do the different machine learning models compare?
6. Can the reported results be reproduced?
7. Are there methodological or reproducibility issues within the project?

1. Project Overview

The purpose of this report is to reproduce and critically evaluate an existing machine learning project for phishing website detection. The selected project is the GitHub repository "*URL-Based Phishing Detection Using Machine Learning*".

The project aims to classify websites as either legitimate or phishing based on a collection of URL and website-related characteristics. These features describe properties such as URL structure, domain information, HTTPS usage, website traffic, anchor URLs, and other indicators commonly associated with phishing attempts.

Phishing is one of the most common cybersecurity threats. Attackers create fraudulent websites that imitate trusted organizations in order to steal sensitive information such as usernames, passwords, and financial credentials. Because phishing websites often exhibit identifiable patterns, machine learning techniques can be used to automatically detect suspicious URLs before users interact with them.

The original project evaluates multiple machine learning algorithms, including Logistic Regression, K-Nearest Neighbors, Naive Bayes, Decision Trees, Random Forests, Support Vector Machines, Multi-Layer Perceptrons, and Gradient Boosting. According to the author, Gradient Boosting achieved the

highest performance with an accuracy of approximately 94.5%, making it the best-performing model among those evaluated.

The objective of this report is not only to reproduce the author's results, but also to examine the quality of the dataset, evaluate the preprocessing and modeling decisions, verify the reported findings, identify potential reproducibility issues, and assess whether the author's conclusions are supported by the experimental evidence.

2. Dataset Description and Initial Exploration

2.1 Dataset Overview

The dataset used in the project contains 11,054 observations and 32 columns. One column serves as an index, one column represents the target variable (class), and the remaining columns represent cybersecurity-related features extracted from URLs and websites. The index column does not appear to contain meaningful cybersecurity information and functions solely as a row identifier. Therefore, its removal before model training is appropriate because it does not contribute predictive value and may introduce unnecessary noise into the analysis. The dataset was obtained from the GitHub repository and originates from a publicly available phishing website dataset commonly used for phishing detection research.

Examples of features include:

- UsingIP
- LongURL
- ShortURL
- Symbol@
- HTTPS
- AnchorURL
- WebsiteTraffic
- GoogleIndex
- StatsReport

Most features are encoded using discrete values such as -1, 0, and 1, representing different levels of legitimacy or suspicion.

The target variable contains two classes:

- **1** = Legitimate Website
- **-1** = Phishing Website

2.2 Missing Values

An inspection of the dataset revealed that no missing values are present. Therefore, no missing-value handling or imputation techniques were required prior to model training.

2.3 Duplicate Records

The original notebook reported that duplicate records were detected and removed before model training. Reproducing this step successfully confirmed the author's finding.

The dataset initially contained 11,054 observations. After applying duplicate removal, the dataset size decreased to 5,849 observations, indicating that 5,205 duplicate rows were present.

This result confirms that duplicate removal was an important preprocessing step in the original workflow and significantly reduced the dataset size before model training. Because nearly half of the observations were duplicates, removing them helps reduce potential bias and prevents certain records from exerting disproportionate influence on model training and evaluation.

2.4 Class Distribution

After duplicate removal, the target variable consisted of the following classes:

Class	Number of Samples
1 (Legitimate)	2,830
-1 (Phishing)	3,019

This corresponds to approximately:

- 48.4% legitimate websites
- 51.6% phishing websites

The class distribution is nearly balanced, with only a small difference between the two classes. This finding is important because the original notebook later applies SMOTE (Synthetic Minority Oversampling Technique) to balance the training data.

Although the class distribution is relatively balanced, the original notebook applies SMOTE to the training data to eliminate any remaining imbalance. The effectiveness and impact of this preprocessing step are discussed later in the report.

3. Exploratory Data Analysis (EDA)

3.1 Feature Characteristics

The phishing dataset contains a collection of manually engineered cybersecurity features designed to describe the properties of URLs and websites. Unlike many machine learning datasets that contain continuous numerical measurements, most variables in this dataset are categorical indicators encoded as discrete values such as -1, 0, and 1.

Examples include:

- UsingIP
- HTTPS
- AnchorURL
- GoogleIndex
- WebsiteTraffic

- StatsReport

These features represent security-related characteristics that may indicate whether a website is legitimate or phishing.

An important observation is that most features are already preprocessed and transformed into security indicators rather than raw website attributes. Therefore, this project focuses primarily on classification rather than feature extraction.

3.2 Data Quality Assessment

The dataset demonstrated good overall quality:

- No missing values were detected.
- A substantial number of duplicate rows (5,205) were detected and removed during preprocessing.
- All features were stored in numeric format.
- No label encoding was required.

This significantly simplifies the preprocessing stage and reduces the risk of introducing errors during data cleaning.

3.3 Initial Concerns Regarding Feature Types

Although the original notebook performs several outlier-detection procedures, including:

- Z-score analysis
- Interquartile Range (IQR) analysis
- Logarithmic transformation

these methods are traditionally intended for continuous numerical variables.

Most variables in this dataset contain only a small set of discrete values (typically -1, 0, and 1). As a result, the concept of an "outlier" becomes less meaningful than it would be in a dataset containing measurements such as income, age, or transaction amounts.

For example, if a feature can only take three possible values, it becomes difficult to justify treating one of those values as an outlier. Because the original features are discrete indicators, the justification for applying traditional outlier-detection methods is less clear than it would be for continuous numerical measurements.

This raises an important methodological question regarding whether the outlier-handling procedures performed in the original notebook were necessary or appropriate for this dataset.

3.4 Class Distribution Analysis

The target variable was found to be relatively balanced:

- Class 1: 2,830 observations (48.4%)
- Class -1: 3,019 observations (51.6%)

The difference between classes is approximately 3.2 percentage points, indicating a moderate rather than severe class imbalance.

While the dataset is not perfectly balanced, the imbalance is relatively mild compared to many cybersecurity datasets where malicious events may represent less than 1% of all observations.

This observation is particularly relevant because the original notebook applies SMOTE oversampling before training the models. Further analysis will be conducted to determine whether this step provided meaningful improvements or unnecessarily altered the data distribution.

3.5 Correlation Analysis

To better understand the predictive power of individual features, a correlation analysis was performed between each feature and the target variable (class).

The following table presents the features exhibiting the strongest positive correlations with the target variable.

Feature	Correlation
HTTPS	0.693
AnchorURL	0.679
PrefixSuffix-	0.381
SubDomains	0.296
WebsiteTraffic	0.282
LinksInScriptTags	0.244
RequestURL	0.238

The results support the author's conclusion that HTTPS, AnchorURL, and WebsiteTraffic are among the most influential features for phishing detection.

In particular, HTTPS and AnchorURL exhibit substantially stronger relationships with the target variable than any other feature in the dataset. This suggests that the security characteristics captured by these variables play a significant role in distinguishing phishing websites from legitimate ones.

In addition to correlation analysis, crosstab analysis was performed for several highly influential features, including HTTPS, AnchorURL, and WebsiteTraffic. The results showed clear differences in feature distributions between phishing and legitimate websites. This finding supports the correlation analysis and provides additional evidence that these features contribute meaningful information for phishing detection.

From a cybersecurity perspective, this result is intuitive. Legitimate websites are more likely to use HTTPS correctly, maintain trustworthy link structures, and exhibit stronger reputation indicators such as higher website traffic. Phishing websites often lack these characteristics, which explains the strong correlations observed.

The correlation analysis partially validates the author's conclusions regarding important predictors. A more detailed evaluation of the feature-selection process is provided in the following section.

Pearson correlation was used to examine the linear relationship between each feature and the target variable. Because most variables are ordinal indicators encoded using values such as -1, 0, and 1, Spearman correlation may be theoretically more appropriate because it is designed for ordinal data and measures monotonic relationships without assuming interval-scaled variables. However, Pearson correlation was used to maintain consistency with the original notebook and to allow direct comparison with the author's results.

It is important to note that correlation does not imply causation. A strong correlation indicates association with the target variable but does not necessarily mean that a feature directly causes a website to be classified as phishing or legitimate.

3.6 Feature Selection Evaluation

The notebook removes several features after examining their correlation with the target variable. While reducing the number of features can simplify a model and potentially reduce overfitting, the feature-selection process appears to rely primarily on visual inspection of correlation values.

Correlation alone is often insufficient to determine a feature's usefulness. Some variables may contribute predictive information when combined with other features, even if their individual correlation with the target variable is relatively small.

For example, the removed feature *DomainRegLen* exhibited a correlation magnitude of approximately 0.226 with the target variable, which exceeds the correlation values of several features that were retained.

A more rigorous feature-selection strategy could include feature importance rankings, recursive feature elimination (RFE), permutation importance, or cross-validation-based feature selection. Such approaches would provide stronger evidence that the removed variables were truly unnecessary.

Therefore, while the feature-selection process reduced dimensionality and produced strong classification performance, the notebook provides limited evidence that all removed features were unnecessary. Additional feature-selection techniques could provide stronger justification for these decisions.

4. Feature Engineering Analysis

4.1 Overview

Feature engineering plays an important role in machine learning because it transforms raw data into representations that can improve model performance. In cybersecurity applications, feature engineering is particularly important because malicious behavior is often identified through patterns embedded within network traffic, URLs, or system activity rather than through a single indicator.

Unlike many machine learning projects that begin with raw data, the phishing dataset used in this project already contains manually engineered cybersecurity features. Most variables are encoded as discrete indicators that describe characteristics of URLs and websites, such as the use of HTTPS, URL structure, domain properties, and website reputation indicators.

As a result, the feature-engineering process in the notebook focused primarily on preprocessing, feature selection, scaling, and class balancing rather than creating entirely new features.

4.2 Feature Selection

The original notebook removes several features after examining their correlation with the target variable. The removed features include:

- LongURL
- ShortURL
- Symbol@
- Redirecting//
- DomainRegLen
- Favicon
- UsingPopupWindow
- IframeRedirection
- LinksPointingToPage

The motivation for removing these variables was to reduce dimensionality and eliminate features that appeared to have weaker relationships with the target variable.

However, correlation alone is often insufficient for determining whether a feature is useful. Some variables may provide predictive value when combined with other features even if their individual correlation appears small.

For example, DomainRegLen exhibited a correlation magnitude of approximately 0.226 with the target variable, which exceeded the correlation values of several retained features. This suggests that potentially useful information may have been discarded.

Although the feature-selection process reduced the number of variables and resulted in strong classification performance, the notebook provides limited evidence that all removed features were unnecessary. In addition to correlation analysis, feature importance scores from the Gradient Boosting model were examined. The feature importance analysis identified HTTPS, AnchorURL, and WebsiteTraffic among the most influential predictors. This result is consistent with the correlation and crosstab analyses, providing additional evidence that these features contribute substantially to phishing website detection performance. However, the feature importance results do not fully validate the removal of all discarded features, and a more systematic feature-selection methodology would provide stronger justification for these decisions.

4.3 Feature Scaling

The notebook applies feature scaling before training several machine learning models.

Feature scaling transforms variables so that they share a similar numerical range. This is particularly important for algorithms that rely on distance calculations or gradient-based optimization, such as:

- K-Nearest Neighbors (KNN)
- Support Vector Machines (SVM)
- Multi-Layer Perceptrons (MLP)

Without scaling, features with larger numerical ranges can dominate the learning process and distort distance calculations.

Although many variables in this dataset are already encoded using small values such as -1, 0, and 1, scaling creates a consistent representation across all features and ensures compatibility with algorithms that are sensitive to feature magnitude. In contrast, tree-based models such as Decision Trees, Random Forests, and Gradient Boosting are generally less sensitive to feature scaling because they split data based on feature thresholds rather than distance calculations.

Therefore, feature scaling is a reasonable preprocessing step for this dataset.

4.4 Class Balancing Using SMOTE

The notebook applies Synthetic Minority Oversampling Technique (SMOTE) to balance the training data before model training.

SMOTE generates synthetic observations for the minority class by interpolating between existing observations. The objective is to reduce class imbalance and improve a model's ability to recognize minority-class examples.

However, after duplicate removal the dataset is nearly balanced:

- Legitimate websites: 48.4%
- Phishing websites: 51.6%

Compared with many cybersecurity datasets, where malicious events may represent only a very small percentage of observations, this imbalance is relatively mild.

As a result, it is not immediately clear whether SMOTE was necessary. A stronger experimental design would compare model performance both before and after oversampling to determine whether SMOTE provides meaningful improvements.

While the use of SMOTE is not inherently incorrect, the notebook does not provide sufficient evidence that oversampling significantly improved model performance.

To further evaluate the impact of SMOTE, an additional Gradient Boosting experiment was conducted using the original training data without oversampling. The model achieved an accuracy of 94.27%, Precision of 93.38%, Recall of 94.25%, and F1-score of 93.81%. After applying SMOTE, the corresponding values increased only slightly to 94.62%, 93.75%, 94.62%, and 94.18%, respectively. These results suggest that SMOTE provided only a modest performance improvement and was not a major factor in the model's success.

4.5 Outlier Handling and Transformations

The notebook performs several preprocessing operations intended to identify and handle outliers, including:

- Z-score analysis
- Interquartile Range (IQR) analysis
- Logarithmic transformations

These techniques are commonly used for continuous numerical variables because they help identify extreme observations that may negatively influence model performance.

However, most variables in the phishing dataset are categorical indicators encoded using values such as -1, 0, and 1. Because these features contain only a small number of possible values, the concept of an outlier is less meaningful than it would be in a dataset containing continuous measurements.

Consequently, the justification for applying traditional outlier-detection techniques is weaker than in many other machine learning applications. Additional explanation would have strengthened the rationale for these preprocessing decisions.

4.6 Redundancy Analysis and Potential Improvements

Feature redundancy occurs when multiple variables capture similar information. Excessive redundancy can increase model complexity without providing additional predictive value.

One way to identify redundancy is through correlation analysis. Features that are highly correlated with one another may contain overlapping information. Additional techniques such as Variance Inflation Factor (VIF), feature importance analysis, or dimensionality-reduction methods could also be used to identify redundant variables. Correlation analysis between features (rather than between features and the target variable) can reveal whether multiple variables are capturing similar information.

The notebook partially addresses redundancy through feature removal, but the analysis is limited to correlations with the target variable rather than correlations among features themselves.

Several additional features could potentially improve phishing detection performance, including:

- Domain age in days
- WHOIS registration information
- SSL certificate details
- Historical blacklist information
- Domain reputation scores
- URL lexical complexity measures

These features may provide additional information about website legitimacy and could potentially improve model performance in real-world phishing detection systems.

5. Reproducibility Analysis

One of the primary goals of this project was to reproduce the results reported in the original GitHub repository. During the reproduction process, several issues were identified that affect the reproducibility and transparency of the project.

5.1 Hardcoded File Paths

The notebook loads the dataset using a hardcoded Windows file path:

```
df = pd.read_csv("C:\\Users\\SUMANJALI\\URL-Based Phishing Detection Using Machine Learning\\phishing.csv")
```

This approach prevents the notebook from running successfully on other machines without modification.

For reproducible research, datasets should be loaded using relative paths, such as:

```
df = pd.read_csv("phishing.csv")
```

or

```
df = pd.read_csv("./data/phishing.csv")
```

Although this issue is easily corrected, it reduces the portability of the implementation and creates an unnecessary barrier for reproduction.

5.2 Documentation and Consistency Issues

Several documentation inconsistencies were identified during the review of the notebook.

Most notably, the "Model Building & Training" section describes the problem as predicting suicide rates and refers to a regression task involving a continuous numerical outcome.

However, the actual project focuses on phishing website classification using a binary target variable.

This strongly suggests that portions of the notebook were copied from a different project and not fully revised before publication.

A second inconsistency appears in the conclusion section, which refers to reducing the risk of malicious attachments despite the project focusing on phishing URLs rather than file attachments.

Although these issues do not directly affect model performance, they reduce the credibility and professionalism of the documentation and may confuse readers attempting to understand the methodology.

In addition to documentation concerns, several methodological decisions in the preprocessing pipeline warrant further examination.

5.3 Preprocessing Justification

The notebook applies several preprocessing techniques, including:

- Duplicate removal
- Outlier detection
- Logarithmic transformation
- SMOTE oversampling
- Feature elimination

While these techniques are common in machine learning workflows, the rationale behind several of these decisions is not always clearly explained.

In particular:

- The dataset consists primarily of categorical indicators encoded as -1, 0, and 1.
- Traditional outlier-detection methods are generally designed for continuous numerical variables.
- The class distribution exhibits only a moderate class imbalance (51.6% phishing versus 48.4% legitimate).

In addition, the notebook applies feature scaling before the train-test split. This introduces data leakage because information from the test set influences the scaling parameters used during training. Although the impact may be limited in this dataset, the resulting performance estimates may be slightly optimistic compared with a fully isolated evaluation procedure.

Consequently, stronger justification should have been provided for some preprocessing choices, particularly outlier handling and oversampling.

5.4 Randomness and Reproducibility

Several machine learning algorithms used in the notebook contain stochastic components, including Decision Trees, Random Forests, and Multi-Layer Perceptrons.

However, the notebook does not consistently specify a `random_state` parameter when initializing these models.

Because several models do not specify a fixed random seed, slight variations in results may occur across different executions. For example:

Model	First Run	Second Run
Random Forest	92.82%	94.10%
MLP	94.02%	93.93%
Decision Tree	91.45%	91.28%

These differences are relatively small, but they demonstrate that the experiments are not fully deterministic.

For reproducible machine learning experiments, fixed random seeds should be specified consistently across all stochastic models, for example:

```
random_state = 42
```

Despite these variations, Gradient Boosting remained the highest-performing model in all observed executions, indicating that the author's primary conclusion is robust.

5.5 Overall Reproducibility Assessment

The project demonstrates a moderate level of reproducibility.

Positive aspects include:

- Publicly available dataset
- Complete notebook implementation
- Multiple machine learning models available for evaluation

However, several issues reduce confidence in exact reproduction:

- Hardcoded dataset paths
- Limited preprocessing documentation
- Documentation errors
- Missing random seeds

Overall, the project can be reproduced successfully and the author's primary findings were verified. However, additional documentation, stronger reproducibility practices, and clearer methodological justification would significantly improve transparency, reliability, and ease of reproduction.

6. Methodology Evaluation and Critical Analysis

6.1 Evaluation of Class Imbalance Handling

The author identifies class imbalance within the dataset and applies SMOTE (Synthetic Minority Oversampling Technique) before training the machine learning models.

SMOTE is a commonly used technique that generates synthetic examples of minority-class observations in order to create a more balanced training dataset.

However, an independent examination of the dataset revealed the following class distribution:

Class	Samples
-------	---------

1	2,830
---	-------

-1	3,019
----	-------

This corresponds to approximately:

- 48.4% legitimate websites
- 51.6% phishing websites

After duplicate removal, the dataset is nearly balanced. As a result, the necessity of applying SMOTE is not immediately obvious.

A stronger methodology would compare:

1. Models trained on the original dataset.
2. Models trained after SMOTE.

This comparison would allow researchers to determine whether oversampling actually improves performance or simply increases model complexity.

An additional experiment was conducted to evaluate the impact of SMOTE. A Gradient Boosting model trained without oversampling achieved an accuracy of 94.27%, compared with 94.62% after

SMOTE was applied. Precision, Recall, and F1-score also improved by less than half a percentage point. These findings suggest that SMOTE provided only a modest benefit and that the strong performance of the model was primarily driven by the predictive quality of the selected features rather than by oversampling. Therefore, while the use of SMOTE is not inherently inappropriate, its overall impact on this relatively balanced dataset appears limited.

6.2 Evaluation of Outlier Detection Procedures

The notebook performs several outlier-related preprocessing operations, including:

- Z-score analysis
- Interquartile Range (IQR) analysis
- Logarithmic transformations

These techniques are traditionally used for continuous numerical variables.

However, the phishing dataset consists primarily of categorical indicators encoded using values such as:

- -1
- 0
- 1

For many features, these values represent categories rather than measurements.

Examples include:

- HTTPS status
- Presence of suspicious symbols
- Domain characteristics

In such cases, applying traditional outlier-detection techniques may not be meaningful because the features contain only a limited number of possible values.

This raises concerns regarding whether the outlier-removal process contributed meaningful improvements to the model performance or simply increased preprocessing complexity.

A more suitable approach would be to justify outlier handling individually for each feature rather than applying generic numerical techniques across the entire dataset.

6.3 Evaluation of Feature Selection

The author removes multiple features after examining correlation values.

Feature selection is an important step because removing irrelevant variables can:

- Reduce overfitting
- Improve training speed
- Increase model interpretability

However, correlation alone is often insufficient for deciding whether a feature should be removed.

The independent correlation analysis performed during this project revealed that some removed features still exhibited moderate relationships with the target variable.

For example, the feature *DomainRegLen* demonstrated a stronger relationship with the target variable than several features that were retained.

This suggests that potentially useful information may have been discarded during preprocessing, although the final model performance remained strong.

More robust feature-selection methods could include:

- Recursive Feature Elimination (RFE)
- Feature importance analysis
- Cross-validation-based feature selection
- Mutual information analysis

Such approaches would provide stronger evidence for feature-removal decisions.

6.4 Evaluation of Performance Metrics

The notebook primarily emphasizes accuracy when comparing machine learning models.

While accuracy is easy to understand, it is not always the most informative metric in cybersecurity applications.

Throughout the course, significant emphasis was placed on the dangers of relying solely on accuracy when evaluating security systems.

In phishing detection systems:

- False negatives may allow malicious websites to remain undetected.
- False positives may incorrectly block legitimate websites.

For this reason, additional metrics such as:

- Precision
- Recall
- F1-score
- Confusion Matrix analysis

are often more informative than accuracy alone.

To the author's credit, the notebook does report confusion matrices and classification reports for all models. However, the discussion and model ranking rely primarily on accuracy, while the additional metrics receive comparatively limited interpretation.

A comprehensive evaluation should consider all of these metrics rather than focusing mainly on overall accuracy.

6.5 Evaluation of the Author's Main Claim

The primary conclusion of the original project is that Gradient Boosting achieved the highest performance and represents the best model for phishing detection among the algorithms evaluated.

The independent analysis largely supports this claim.

The reproduction process confirmed that Gradient Boosting achieved the highest classification accuracy (94.62%) and produced strong Precision (93.75%), Recall (94.62%), F1-score (94.18%), MCC (0.892), and ROC-AUC (0.992) values.

Furthermore, correlation analysis confirmed that several features highlighted by the author, including:

- HTTPS
- AnchorURL
- WebsiteTraffic

are indeed among the strongest predictors in the dataset.

Therefore, the author's conclusions regarding important predictive features appear justified.

However, several methodological limitations reduce confidence in the strength of the supporting evidence:

- Limited justification for preprocessing decisions.
- Heavy reliance on accuracy as the primary comparison metric.
- Lack of comparison between original and SMOTE-balanced datasets.
- Insufficient explanation of feature-removal decisions.
- Absence of fixed random seeds for stochastic models.

Consequently, while the final results appear reasonable and the primary conclusion is supported, certain aspects of the methodology could be strengthened to provide greater confidence in the reported findings.

7. Model Reproduction and Results

7.1 Overview

The notebook evaluates eight machine learning algorithms for phishing website detection:

1. Logistic Regression
2. K-Nearest Neighbors (KNN)
3. Naive Bayes
4. Decision Tree
5. Random Forest
6. Gradient Boosting

7. Multi-Layer Perceptron (MLP)
8. Support Vector Machine (SVM)

Each model was trained using the same training and testing datasets and evaluated using accuracy, confusion matrices, precision, recall, and F1-score.

7.2 Model Comparison

The final comparison section of the notebook ranked the models according to their classification accuracy.

Rank	Model	Accuracy
1	Gradient Boosting	94.62%
2 (tie)	Random Forest	93.93%
2 (tie)	Multi-Layer Perceptron	93.93%
4	Support Vector Machine	93.85%
5	Logistic Regression	92.39%
6	Decision Tree	92.14%
7	K-Nearest Neighbors	91.79%
8	Naive Bayes	66.15%

All models except Naive Bayes achieved accuracies above 90%, indicating that the selected feature set contains substantial predictive information for phishing detection.

The results indicate that ensemble-based methods achieved the strongest performance on this dataset. Gradient Boosting and Random Forest occupied the top two positions, while simpler models such as Logistic Regression and K-Nearest Neighbors achieved slightly lower accuracy. Naive Bayes performed substantially worse than all other evaluated models.

The difference between the top-performing models was relatively small. Gradient Boosting achieved an accuracy of 94.62%, while Random Forest, Multi-Layer Perceptron, and Support Vector Machine all achieved accuracies above 93.8%. This suggests that several machine learning algorithms are capable of effectively detecting phishing websites using the selected feature set, although Gradient Boosting produced the strongest overall results.

7.3 Best Performing Model

Gradient Boosting achieved the highest overall performance with an accuracy of 94.62%.

Confusion Matrix:

```
[[597, 34],  
 [29, 510]]
```

The model achieved balanced precision and recall values across both classes, indicating that it successfully identified phishing and legitimate websites without exhibiting a strong classification bias toward either category.

The model achieved strong Precision, Recall, and F1-score values, further supporting its effectiveness.

From a cybersecurity perspective, this performance indicates that the model is capable of detecting most phishing websites while maintaining a relatively low rate of false alarms for legitimate websites.

The relatively low number of false positives and false negatives suggests that the model maintained a good balance between identifying phishing websites and avoiding the misclassification of legitimate websites.

Additional evaluation metrics further confirmed the effectiveness of the model. Gradient Boosting achieved a Precision of 93.75%, Recall of 94.62%, F1-score of 94.18%, Matthews Correlation Coefficient (MCC) of 0.892, and a ROC-AUC score of 0.992. These results indicate excellent discrimination between phishing and legitimate websites and demonstrate strong overall classification performance.

7.4 Worst Performing Model

Naive Bayes produced the lowest performance with an accuracy of only 66.15%.

Confusion Matrix:

```
[[631, 0],  
 [395, 144]]
```

Although the model correctly classified all observations belonging to Class -1, it correctly identified only a small proportion of Class 1 observations. This resulted in a highly unbalanced classifier that favored one class while performing poorly on the other. The results illustrate why accuracy alone can be misleading and highlight the importance of examining precision, recall, and F1-scores alongside overall accuracy.

7.5 Reproduction Findings

The reproduced results strongly support the findings reported by the author. The reproduced Gradient Boosting model achieved an accuracy of 94.62%, which is extremely close to the performance reported in the original notebook. In addition, the overall ranking of the evaluated models was highly consistent with the author's results.

However, small variations may occur between different executions of the notebook for models such as Random Forest, Decision Tree, and MLP because fixed random seeds were not consistently specified.

Most importantly, Gradient Boosting remained the highest-performing model in every observed execution. This consistency strengthens confidence in the author's primary conclusion despite the reproducibility issues identified earlier.

7.6 Evaluation Metrics Interpretation

Several evaluation metrics were used to assess model performance.

Accuracy measures the proportion of correctly classified observations:

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

In phishing detection, accuracy indicates the overall percentage of websites that were correctly classified as either phishing or legitimate. However, accuracy alone may be misleading because it does not distinguish between different types of classification errors.

Precision measures the proportion of websites predicted as phishing that were actually phishing:

$$\text{Precision} = TP / (TP + FP)$$

In a cybersecurity context, high precision reduces the number of false alarms and minimizes the incorrect blocking of legitimate websites.

Recall measures the proportion of actual phishing websites that were correctly detected:

$$\text{Recall} = TP / (TP + FN)$$

Recall is particularly important in phishing detection because false negatives allow malicious websites to evade detection and potentially compromise users.

F1-Score combines Precision and Recall into a single metric:

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

The F1-score is useful when both false positives and false negatives are important and a balance between Precision and Recall is desired.

Matthews Correlation Coefficient (MCC) provides a balanced evaluation that considers all elements of the confusion matrix:

$$MCC = (TP \times TN - FP \times FN) / \sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}$$

MCC is particularly useful for cybersecurity applications because it remains informative even when class distributions are imbalanced.

ROC-AUC (Receiver Operating Characteristic – Area Under the Curve) measures the model's ability to distinguish between classes across different classification thresholds.

A ROC-AUC value of 1.0 represents perfect discrimination, while 0.5 indicates random guessing. The reproduced Gradient Boosting model achieved a ROC-AUC score of 0.992, indicating excellent separation between phishing and legitimate websites.

Confusion Matrix summarizes classification outcomes using True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN).

In phishing detection, false negatives are generally considered the most dangerous errors because they allow malicious websites to bypass detection, while false positives may incorrectly classify legitimate websites as phishing.

8. Error Analysis

8.1 False Positives and False Negatives

In phishing detection, two types of classification errors are particularly important:

- **False Positives:** Legitimate websites incorrectly classified as phishing.
- **False Negatives:** Phishing websites incorrectly classified as legitimate.

False negatives are generally more dangerous because they allow malicious websites to bypass detection and potentially steal sensitive user information. False positives can also be problematic because they may block access to legitimate websites and reduce user trust in the detection system.

8.2 Error Patterns

The confusion matrix of the best-performing model (Gradient Boosting) indicates that both error types occurred, although at relatively low levels.

Confusion Matrix:

```
[[597, 34],  
 [29, 510]]
```

The model produced 34 false positives and 29 false negatives. The relatively balanced distribution of errors suggests that the classifier did not strongly favor one class over the other.

8.3 Cybersecurity Implications

From a cybersecurity perspective, minimizing false negatives is particularly important because undetected phishing websites may expose users to credential theft, financial fraud, and other security risks.

At the same time, excessive false positives can reduce the usability of the system by incorrectly flagging legitimate websites as malicious.

The results obtained by the Gradient Boosting model suggest a reasonable balance between security and usability, making it suitable for phishing detection scenarios where both detection accuracy and user experience are important considerations.

9. Discussion

The objective of this review was not only to reproduce the results of the project but also to critically evaluate the methodology and conclusions presented by the author.

Several strengths were identified throughout the analysis. The project uses a publicly available dataset, evaluates multiple machine learning algorithms, performs exploratory data analysis, and reports several performance metrics. The final conclusion that Gradient Boosting achieved the highest performance was supported by the reproduced results. Furthermore, the independent analysis confirmed that important features identified by the author, such as HTTPS, AnchorURL, and WebsiteTraffic, are among the strongest predictors in the dataset, with HTTPS and AnchorURL exhibiting particularly strong correlations with the target variable.

At the same time, several weaknesses were identified. The notebook contains hardcoded file paths that prevent direct execution on other systems. In addition, portions of the documentation appear to

have been copied from unrelated projects, including references to suicide-rate prediction and malicious attachments, despite the project focusing on phishing URL classification.

The preprocessing methodology also lacks sufficient justification in several areas. Techniques such as duplicate removal, outlier detection, scaling, and SMOTE oversampling were applied, but the reasoning behind these decisions was not always clearly explained. Although these choices do not necessarily invalidate the results, additional justification would improve confidence in the methodology.

Another limitation is the inconsistent use of model hyperparameter tuning. Some algorithms, such as Gradient Boosting and Decision Trees, received custom parameter settings, while other models were evaluated primarily using default configurations. As a result, the comparison may not fully represent the maximum achievable performance of every algorithm.

Finally, the absence of fixed random seeds reduces experimental reproducibility and results in small performance differences across repeated executions. Despite these limitations, Gradient Boosting achieved the highest reproduced accuracy (94.62%) and strong Precision, Recall, F1-score, MCC, and ROC-AUC values, indicating that the author's primary conclusion is robust.

Overall, the project demonstrates a reasonable machine learning workflow and produces strong classification results. The reproduction process successfully verified the author's main findings, particularly the effectiveness of Gradient Boosting for phishing website detection. However, additional documentation, reproducibility controls, and methodological justification would improve the quality, transparency, and reliability of the study.

10. Conclusion

This report reproduced and critically evaluated a machine learning project focused on phishing website detection using URL-based features.

The analysis confirmed that the dataset contains informative features capable of distinguishing phishing websites from legitimate websites with high accuracy. Multiple machine learning algorithms were evaluated, with Gradient Boosting achieving the highest overall performance.

The reproduction process successfully verified the author's primary claim that Gradient Boosting is the most effective model among the evaluated algorithms. The reproduced Gradient Boosting model achieved an accuracy of 94.62%, a Precision of 93.75%, a Recall of 94.62%, an F1-score of 94.18%, an MCC of 0.892, and a ROC-AUC score of 0.992.

Although the main findings were validated, several issues affecting reproducibility and documentation were identified. These include hardcoded file paths, inconsistent notebook documentation, missing random seeds, and limited justification for some preprocessing decisions.

Overall, the project demonstrates an effective application of machine learning techniques to the problem of phishing website detection. The author's primary conclusions were supported by the reproduced results, and the selected feature set proved highly informative for classification. Nevertheless, improvements in documentation, reproducibility practices, and methodological justification would strengthen the reliability, transparency, and scientific rigor of the work.